

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности
Кафедра «Информационная безопасность компьютерных систем»

ЛАБОРАТОРНАЯ РАБОТА № 4

«ОРГАНИЗАЦИЯ АНАЛОГОВОГО ВВОДА-ВЫВОДА: АЦП И ШИМ»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр. 5131001/20003

<подпись>

Черникова В.М.

Преподаватель

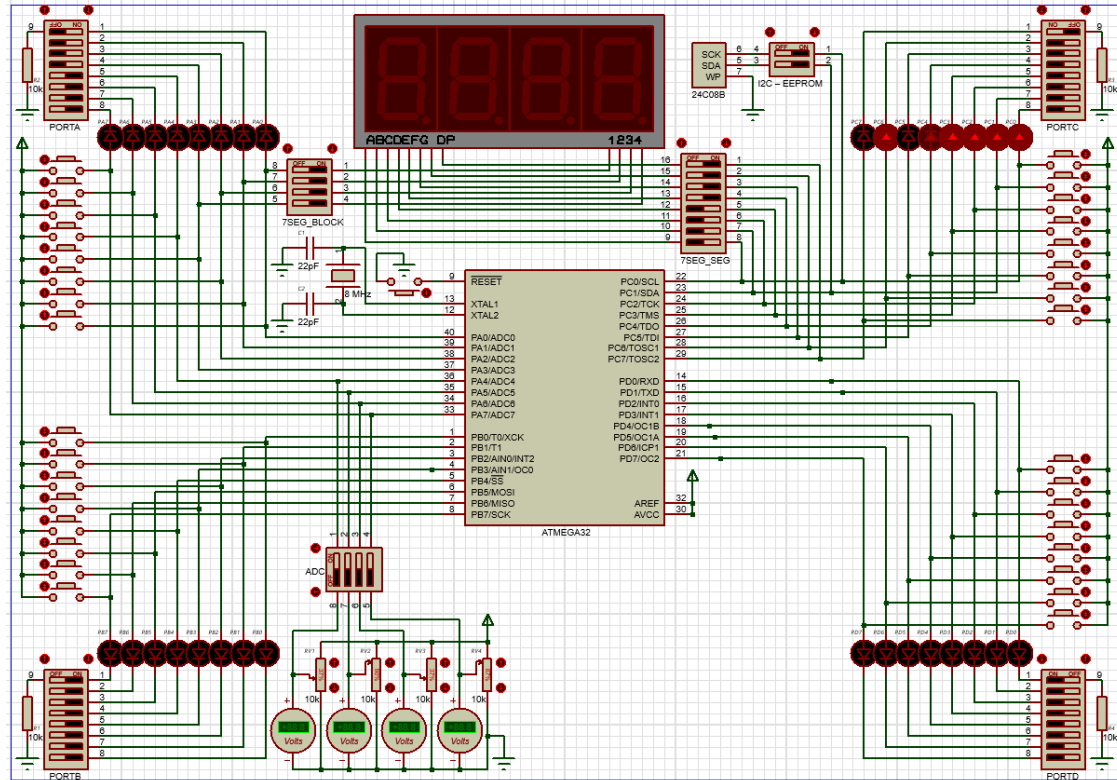
<подпись>

Семенов П.О.

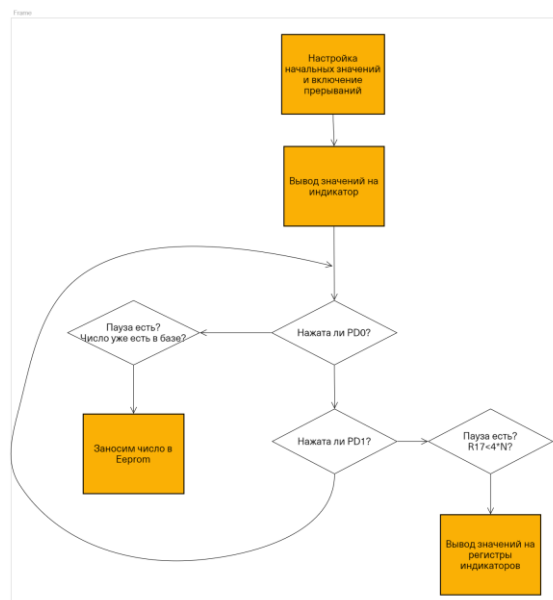
Санкт-Петербург
2024

1. Цель лабораторной работы — Получение практических навыков по работе с аналого-цифровым преобразователем и таймерами-счётчиками. Изучение принципов применения широтно-импульсной модуляции в современных цифровых устройствах.

2. Схема установки (задействованные узлы отладочной платы).



3. Блок-схема алгоритма работы программы



4. Комментированный листинг программы.

```
#include <avr/io.h>
#include <avr/interrupt.h>

char mode = 0; //0 - настройка, 1 - гирлянда
char timer0_flag=0; //0 - горят все , 1 - горят паармтеры
char timer0_counter = 0; // - перемещение лампочек
char i = 1; //текущий PDA0-PDA3
unsigned char h_ = 0b11110100;
unsigned char high_adc;
unsigned char minus = 0b00111111;
unsigned char R_h = 0b00000000 ;
unsigned char ports[3]={0b00000001,0b00000000,0b00000000};
unsigned char buf_port[3]={0b00000001,0b00000000,0b00000000};
unsigned char buf;
int h=0;

void init() {
    PORTC = 0x00;
    DDRC = 0xFF;
    PORTD = 0x00;
    DDRD = 0xF0;
    DDRB = 0xFF;
    PORTB = 0x00;
    PORTA = 0x00;
    DDRA = 0xDF; //DF; // почему так??? не 7F = 0b01111111

    GICR = 0xC0;
    MCUCR = 0b00001010;
    GIFR = 0x00;
}

void timer_init() {
    TIMSK = (1<< OCIE0); //включаем timer0
    TCCR0 = (3<< CS0); // clk/64    8000/64=125, 125*4раза=500мс
    OCR0 = 200;
}

void adc_init() {
    ADCSRA = (1<<ADEN)|(1<<ADIF)|(1<<ADATE)|(1<<ADSC) // Разрешение использования АЦ
    |(1<<ADPS2)|(1<<ADPS1); //|(0<<ADPS0); //Делитель 128 = 64 кГц
    ADMUX = (1<<MUX2)|(1<<MUX0)|(1<<ADLAR); //настройка PA5 + считывание только из ADCH
}

ISR(ADC_vect) {
    high_adc = (unsigned char)ADCH;
    OCR2 = 255 - high_adc;
    OCR1BL = high_adc;

    if ( high_adc<=109) {
        minus = 0b01000000;
        h=-1;
    }
    else {
        minus = 0b00000000;
        h=1;
    }
    if (( high_adc<28)|| (high_adc>227)){ //шаг деления 255/9
        R_h = 0b01100110; //-4 or 4
        h=h*4;
    }
}
```

```
}  
else if ((high_adc>28 && high_adc<=56) || (high_adc>199 && high_adc<=227)) {  
    R_h = 0b01001111; //-3 or 3  
    h=h*3;  
}  
else if ((high_adc>56 && high_adc<=84)|| (high_adc>171 && high_adc<=199)){  
    R_h = 0b01011011; //-2 or 2  
    h=h*2;  
}  
else if ((high_adc>84 && high_adc<=112)|| (high_adc>143 && high_adc<=171)){  
    R_h = 0b00000110; //-1 or 1  
}  
else{  
    R_h = 0b00111111; //0  
    h=0;  
}  
  
}  
ISR(INT0_vect){  
    timer0_counter=0;  
    ports[0]=0b00000001;//возвращение к начальному смещению  
    ports[1]=0b00000000;  
    ports[2]=0b00000000;  
    mode = 1 - mode;  
    if (mode==0){  
        timer_init();  
    }  
    else{  
        TCCR0 = (5<< CS00); //Делитель 1024  
        OCR0 = 255;  
    }  
}  
  
//Прерывания по таймеру, позволяют гореть настройкам  
ISR(TIMER0_COMP_vect){  
    if (mode==0){  
        PORTA = 0x00;  
        timer0_counter++;  
        if (timer0_counter==160){  
            timer0_counter=0;  
            timer0_flag=1-timer0_flag;  
        }  
        i+=1;  
        i%=5;  
  
        if ( timer0_flag==0 && i==1){  
            PORTC = R_h;  
            PORTA = 1;  
        }  
        if ( timer0_flag==0 && i==2){  
            PORTC = minus;  
            PORTA = 2;  
        }  
        if ( i==4){  
            PORTC = h_  
            PORTA = 8;  
        }  
    }  
    else{  
        //--------------------------------алгоритм-----
```

```

timer0_counter++;
if(timer0_counter%10==0){

    PORTA = ports[0];
    PORTB = ports[1];
    PORTC = ports[2];
    if (h>0){
        buf = (buf_port[2] << h) | (buf_port[1] >> (8 - h));
        ports[2] = buf;
        buf = (buf_port[0] << h) | (buf_port[2] >> (8 - h));
        ports[0] = buf;
        buf = (buf_port[1] << h) | (buf_port[0] >> (8 - h));
        ports[1] = buf;
    }
    else if(h<0){
        buf = (buf_port[0] >> -h) | (buf_port[1] << (8+h));
        ports[0] = buf;
        buf = (buf_port[2] >> -h) | (buf_port[0] << (8+h));
        ports[2] = buf;
        buf = (buf_port[1] >> -h) | (buf_port[2] << (8+h) );
        ports[1] = buf;
    }
    buf_port[0]=ports[0];
    buf_port[1]=ports[1];
    buf_port[2]=ports[2];
}

if(timer0_counter==60){
    timer0_counter=0 ;
}
//-----алгоритм конец-----
}

}
int main(void)
{
    init();
    timer_init();
    adc_init();
    sei();
    while(1) {
    }
}

```

5. Ответы на контрольные вопросы

1) Посредством каких регистров производится конфигурирование таймера? Посредством каких регистров производится настройка АЦП?

Настройка АЦП производится посредством регистров:

ADCSRA – регистр управления АЦП;

ADMUX – регистр, задающий входной контакт PORTA для подключения АЦП, ориентирование результата и выбор опорной частоты.

2) В каких режимах может работать АЦП?

АЦП может работать в двух режимах:

- режим «непрерывного измерения», когда АЦП постоянно производит преобразование и обновляет регистры ADCH и ADCL новыми значениями;
- режим единичного преобразования, когда выставляется бит ADSC в регистре ADCSRA; при завершении преобразования он сбрасывается.

3) Какие порты и разряды портов микроконтроллера ATmega32 могут обрабатывать входящие аналоговые сигналы?

Входящие аналоговые сигналы могут обрабатывать все разряды порта А и разряды 2 и 3 порта В.

4) Какими способами реализуется ШИМ?

Для генерации ШИМ-сигнала используется в основном два способа:

- применение аналогового компаратора, на один вход которого подаётся вспомогательный опорный пилообразный или треугольный сигнал значительно большей частоты, чем частота модулирующего сигнала, а на другой — модулирующий непрерывный аналоговый сигнал;
- применение таймеров-счётчиков со специальными настройками.

5) Как настроить ШИМ с помощью таймера-счётчика?

Рассмотрим настройку ШИМ с помощью второго таймера-счётчика ATmega32. Необходимо настроить регистр TCCR2:

Bit	7	6	5	4	3	2	1	0	
	FOC2	WGM20	COM21	COM20	WGM21	CS22	CS21	CS20	TCCR2
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Рисунок 4. Регистр TCCR2.

С помощью битов WGM21:0 нужно выбрать один из двух режимов работы ШИМ: режим ШИМ с корректной фазой (Phase Correct PWM Mode) (0:1) или быстрый ШИМ (Fast PWM) (1:1). С помощью битов COM21:0 нужно выбрать поведение вывода OC2: сброс при совпадении при счёте от нуля и установка при совпадении при счёте к нулю (1:0) и наоборот (1:1) для ШИМ с корректной фазой; сброс при совпадении и установка после переполнения (1:0) или наоборот (1:1) для быстрого ШИМ.